Get unlimited access to the best of Medium for less than $1/week. Become a member

Unstructured PDF Text Extraction



Khadija Mahanga · Follow
7 min read · Mar 27, 2024

Listen Share ••• More

Have you ever encountered a daunting task of extracting content from unstructured PDF files, only to find that existing Python packages fall short of meeting your requirements? By unstructured, I mean PDFs with multiple elements and varying page layouts. Picture a file containing a table on one page, two tables on another, or perhaps none at all. Imagine encountering single-column layouts on one page, and two or three-column layouts on another, with tables nestled within those columns. This was precisely the challenge I faced.

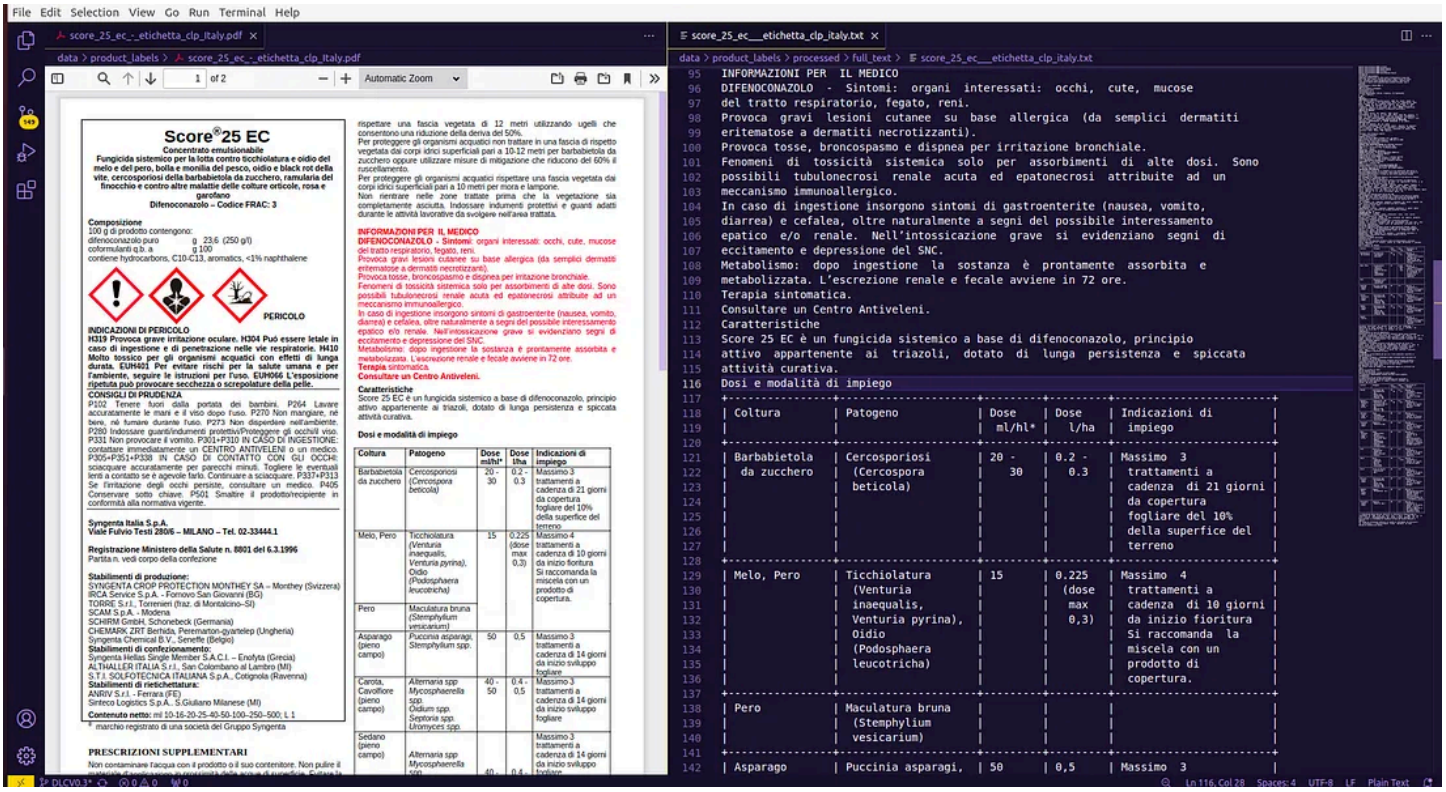
I experimented with various packages like `tabula`, among others. However, each had its strengths and weaknesses. While tabula excelled at extracting tables, it struggled with extracting merged cells information from tables and is not for text extraction. I discovered that `pdfminer` was excellent for extracting text from pages with multiple columns, while `pdfplumber` shone when it came to extracting tables with merged cells. I even found a package called `unstructured`, which was a step in the right direction, meeting about 80% of my needs. Although it performed well in extracting some tables, it often missed others entirely. This issue was common among extraction packages utilizing Optical Character Recognition (OCR), where not all tables could be accurately extracted from PDF files, sometimes even omitting the last row or column of a table.

So, I devised a solution that successfully handled the majority of my sample PDFs. However, I'll address the few exceptions later.

My approach involved combining `pdfminer`, `pdfplumber`, and `fitz` from `pymupdf` to extract text, table information, and images while preserving their layout and flow.

Extracting text and tables

To extract both text and tables in an organised manner, I used pdfplumber package function to identify and extract tables from a page. Subsequently, I utilized a pdfminer function to extract text elements sequentially from the page, excluding those within the boundaries of identified tables. When printing the output string, I ensured to preserve the reading flow and the position of tables.



A screenshot of a sample 2 column layout page of a PDF file and its output string in a text file. **Note: The text file only shows part of the extraction, which is the second layout of the PDF — starting from the red text.**

Let us dive straight into the function without dwelling on packages installation:

In the initial phase of my function, I instantiated objects from each package to process a single PDF file comprehensively. Thereafter, I iterated through each page, prioritizing the identification of table objects that exist in a page using the pdfplumber's `find_tables` function as seen in the code below.

```
def pdf_process(path):
    plumberObj = pdfplumber.open(path)
    minerPages = extract_pages(path)
```

```
        fitzDoc = fitz.open(path)

        ...
        for i in enumerate(minerPages):
            tables = plumberPage.find_tables(table_settings={"text_vertical_ttb": F
            page_text = miner_extract_page(page_layout, tables)
```

Once I had the table objects, I moved to `pdfminer`. While `pdfminer` isn't flawless in table extraction, it excels in handling different page layouts in a PDF. And note my sample PDF files had inconsistent layouts throughout its pages. Therefore, for each page layout, I ensured to crop out all table objects from that page using the helper check function `is_obj_in_bbox`

> *Note, that table object includes a property for it's boundary box called _bbox. We use that boundary in the helper function.*

```
def is_obj_in_bbox(obj, _bbox, page_height):
    """
    checks if an element boundary box is within another boundary
    """
    objx0, y0, objx1, y1 = obj
    x0, top, x1, bottom = _bbox
    return (objx0 >= x0) and (objx1 <= x1) and (page_height - y1 >= top) and (p
```

You will notice here that I am employing `page_height` in the check function and this is due to differences in boundary dimensions between `pdfplumber` package and `pdfminer` package. Read more about the bbox inversion between this two packages from this issues.

Now, let's examine the `miner_extract_page` function:

```
def miner_extract_page(page_layout, tables):
    """
    this function extract texts, tables, images from a single page layout
    Paremeter:
      page_layout -> A pdfminer page object
```

```
        tables -> An array of pdfplumber table objects
    Returns:

    """
    page_height = page_layout.height
    extractedTables = []
    page_output_str = ""

    for element in page_layout:
        if isinstance(element, LTTextContainer):
            tabBox = []
            # if current element exists in any of the tables,
            # append the t to tabBox
            for t in tables:
                is_obj_n_box = is_obj_in_bbox(element.bbox, t.bbox, page_height
                if is_obj_n_box:
                    tabBox.append(t)

            # if tabBox is empty, extract the element with get_text() function
            if not len(tabBox):
                if isinstance(element, LTTextContainer):
                    elementText = element.get_text()
                    page_output_str += elementText
                else:
                    # check for figures layout at this point
                    # check for part 2 when I talk about images/figures

            # else, element exist in a certain table
            # therefore, we extract the found table
            # using pdfplumber table extract function and
            # concatenate to our end results
            else:
                if not tabBox[0] in extractedTables:
                    table_str = tabulate(tabBox[0].extract(**({"vertical_ttb":
                    page_output_str += table_str
                    page_output_str += "\n"
                    # to avoid repetition we used extractedTables to
                    # filter already extracted tables.
                    extractedTables.append(tabBox[0])
    return page_out_str
```

In the above function, I am using tabulate package for printing purposes on the final string. Otherwise it is not necessary. And in case you're extracting a PDF for LLM models, its better to avoid it as it adds more characters to your final output. But also make sure you employ some sort of an organised way to print it to avoid confusion as the output of a table extraction is 2 dimensional array.

Extracting Images

This task also involved identifying certain pictograms/images with the PDF files. I had about 10 different pictograms that I was supposed to check if any of them existed in the file or not. My solution was to first extract all images on the PDF page, and utilize image hashing or other template comparison methods (*matchTemplate from cv2 package*) to determine if the extracted images for the PDF file matched any of the pictograms I had.

One effective package for extracting images is PyMuPDF (fitz), which I chose to utilize. However, I encountered a challenge where not all images were being properly identified or extracted. I speculated whether this was due to the types of images used during the PDF compilation process, such as PNGs or SVGs, but I couldn't confirm this definitively.

For the image extraction process, I implemented a function called `check_for_image`:

```
def check_for_image(pdf_path):
    """
    Function that get images from pdf pages and save them to local drive

    Parameters:
    pdf_path -> Path of the PDF file
    """

    pdf_document = fitz.open(pdf_path)
    xreflist = []

    page_num = 0
    il = pdf_document.get_page_images(page_num)
    logger.info(f"Found {len(il)} images")

    for img in il:
        xref = img[0]
        if xref in xreflist:
            continue

        width = img[2]
        height = img[3]

        #skip tiny images
        if min(width, height) <= 5:
            continue

        imgdata = img["image"]

        imgfile = os.path.join(f"drawing-img-{xref}-{page_num + 1}.{img['ext']}
        fout = open(imgfile, "wb")
```

```
            fout.write(imgdata)
            fout.close()
            xreflist.append(xref)
        pdf_document.close()
```

But because of the image challenge mentioned, I also implemented an extra function in addition to checking normal images. The function was to check for drawings. For drawing can be anything from a lines to more complex shapes (you may read more on the function get_drawings), I employed various steps such as enlarging the rectangle size by a certain number, applying filters for anything smaller or any drawing within a bigger drawings, before getting a `pixmap` of the enlarged area of page that include a drawing in it. I would subsequently compare the pixmap generated to the pictograms I have to make a decision.

```
def check_for_drawings(pdf_path):
    """
    Function that finds drawings from pdf pages and save pixmap of
    on enlarged drawing's rectangle to local drive

    Parameters:
    pdf_path -> Path of the PDF file
    """

    pdf_document = fitz.open(pdf_path)

    for page_num in range(pdf_document.page_count):
        page = pdf_document[page_num]

        d = page.get_drawings()
        new_rects = []
        for p in d:
            # filter emplty rectangle
            if p["rect"].is_empty:
                continue
            w = p["width"]
            if w:
                r = p["rect"] + (-w, -w, w, w)  # enlarge each rectangle by wic
                for i in range(len(new_rects)):
                    if abs(r & new_rects[i]) > 0:  # touching one of the new re
                        new_rects[i] |= r  # enlarge it
                        break

                # now look if contained in one of the new rects
                remainder = [s for s in new_rects if r in s]
                if remainder == []:  # no ==> add this rect to new rects
```

```
                    new_rects.append(r)

        new_rects = list(set(new_rects))  # remove any duplicates
        new_rects.sort(key=lambda r: abs(r), reverse=True)
        remove = []
        for j in range(len(new_rects)):
            for i in range(len(new_rects)):
                if new_rects[j] in new_rects[i] and i != j:
                    remove.append(j)
        remove = list(set(remove))
        for i in reversed(remove):
            del new_rects[i]
        new_rects.sort(key=lambda r: (r.tl.y, r.tl.x))  # sort by location


        mat = fitz.Matrix(5, 5)  # high resolution matrix
        for i, r in enumerate(new_rects):
            if r.width is None or r.height <= 15 or r.width <= 15:
                continue  # skip lines and empty rects
            pix = page.get_pixmap(matrix=mat, clip=r)
            hayPath = f"drawing-rect{page_num}-{i}.png"
            if pix.n - pix.alpha >= 4:       # can be saved as PNG
                pix = fitz.Pixmap(fitz.csRGB, pix)
            pix.save(hayPath)
            pix = None                       # free Pixmap resources

    pdf_document.close()
```

This comprehensive approach allowed me to successfully extract information from nearly all of the sample PDFs I encountered.

In reflecting on the limitations or edge cases of my solution, one challenge I faced was extracting tables that are border-less. Despite extensive efforts, I haven't yet found a straightforward way to achieve this especially if you're dealing with large set of PDF files that needs to be extracted. However, it's worth noting that there are issues and discussions on pdfplumber github pages on the case of table borders. You may find all them using the following search query link https://github.com/jsvine/pdfplumber/issues?q=border

In the meantime, I encourage exploration of open-source projects that may provide valuable insights and potential solutions to this particular challenge.

Text Extraction | Preprocessing Techniques | Python | Pdfplumber | Pdfminer



Follow

Written by Khadija Mahanga

77 Followers

Data Science Enthusiast

More from Khadija Mahanga



Khadija Mahanga

Using Latitude and Longitude data in my machine learning problem.

Early this week, I faced a machine learning problem that had Geographic data in form of coordinates (Latitude and Longitude). I could not

Open in app ↗

Search





Khadija Mahanga

Introduction to Reinforcement Learning with AWS DeepRacer

I recently enrolled myself into the AWS DeepRacer course in Udacity, a course that guides student on how to use reinforcement learning and...

4 min read · Oct 19, 2019

15



Khadija Mahanga

Learning Data Preparation with Kaggle

In a previous story, I mentioned how Kaggle platform was intimidating for a beginner. There are a lot of competitions going on that might...

6 min read · Sep 22, 2019

27



Khadija Mahanga

Want to write something on tech!

I am a software developer, currently working as a Technologist at Code for Africa's Dar-es-salaam lab and I have decided to write. And here...

3 min read · Sep 21, 2018

24

See all from Khadija Mahanga

Recommended from Medium



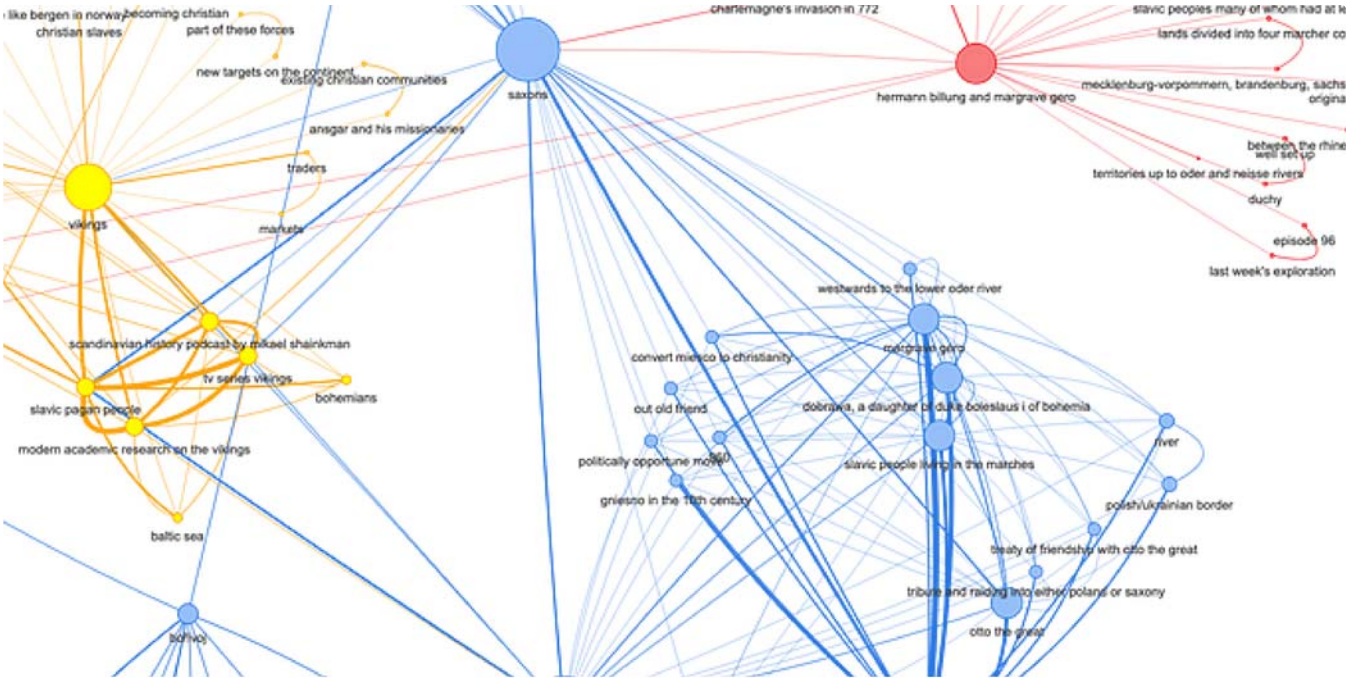
PyMuPDF

Building a RAG Chatbot GUI with the ChatGPT API and PyMuPDF

Create a chatbot for a web browser that can interact with documents

5 min read · Apr 5, 2024

55 1



syrom

Knowledge Graph Extraction & Visualization with local LLM from Unstructured Text: a History example

Motivation an context

19 min read · Apr 1, 2024

400 5

Lists



Coding & Development

11 stories · 559 saves



Predictive Modeling w/ Python

20 stories · 1092 saves



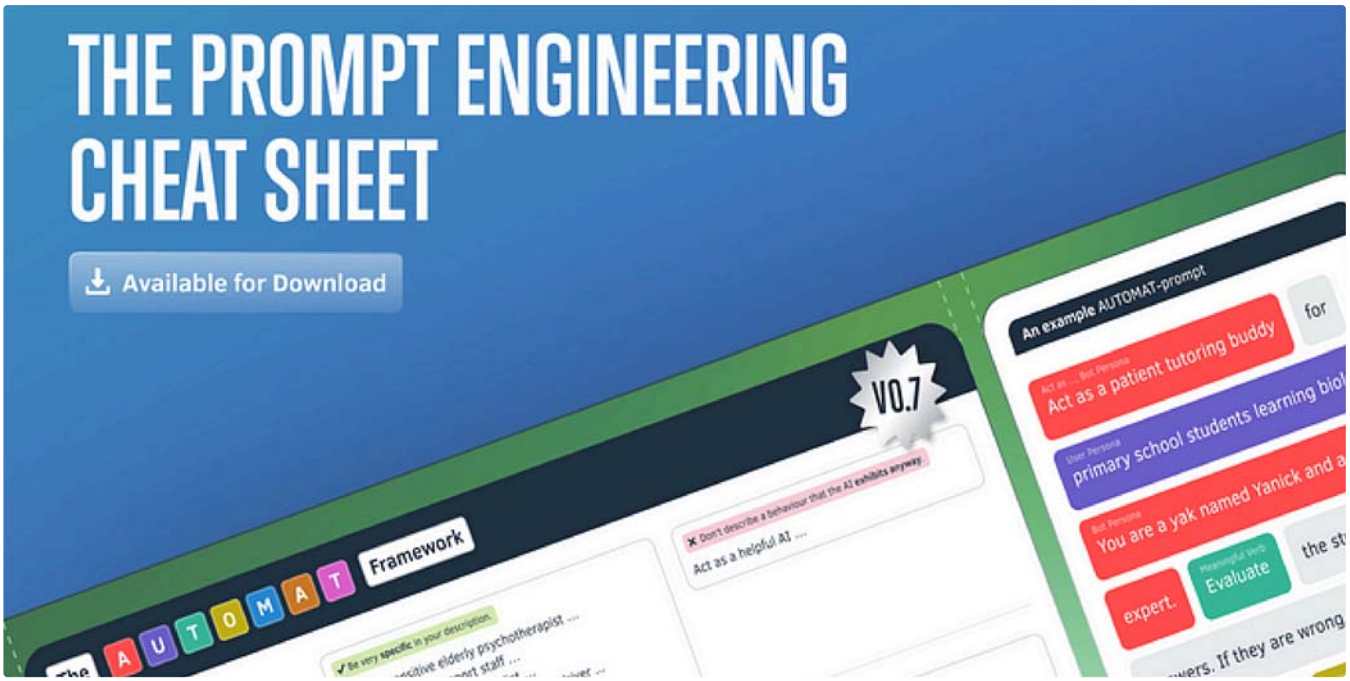
Practical Guides to Machine Learning

10 stories · 1311 saves



ChatGPT

21 stories · 571 saves



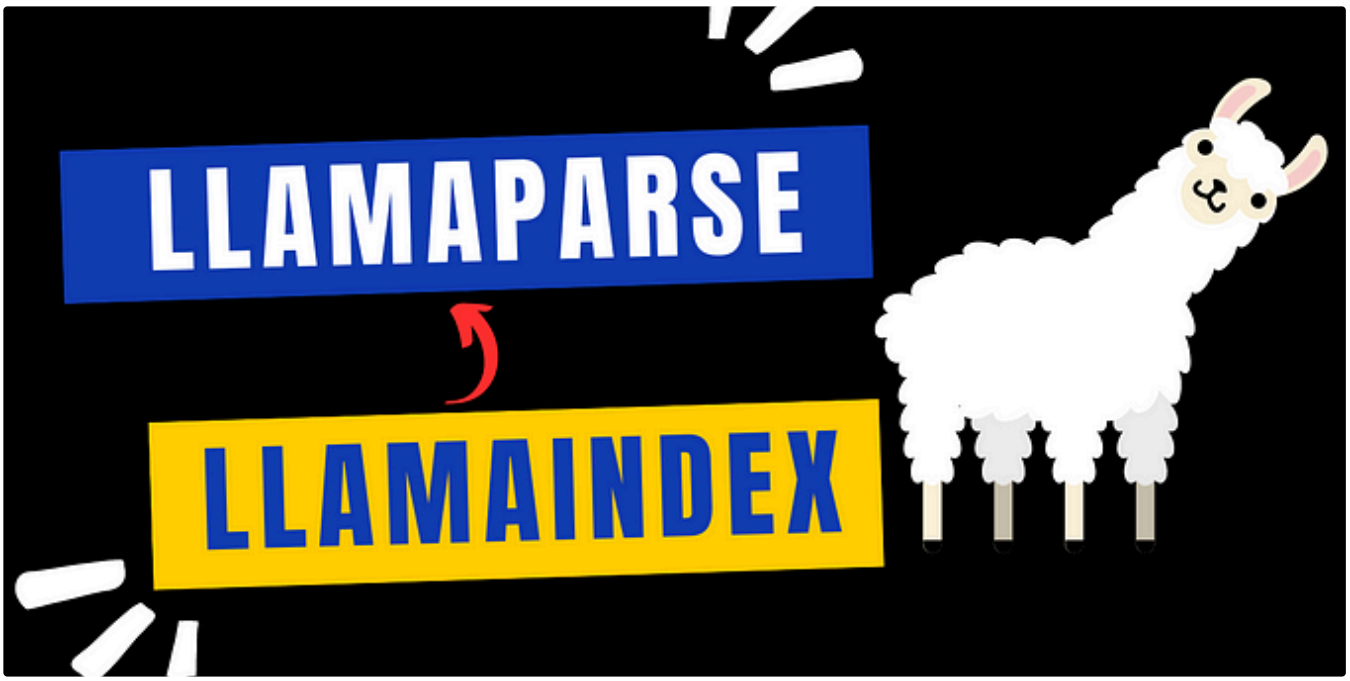
Maximilian Vogel in The Generator

The Perfect Prompt: A Prompt Engineering Cheat Sheet

Large language models can produce any sequence of characters. Literally any. In any idiom, data format or programming language.

7 min read · Apr 8, 2024

1K 6



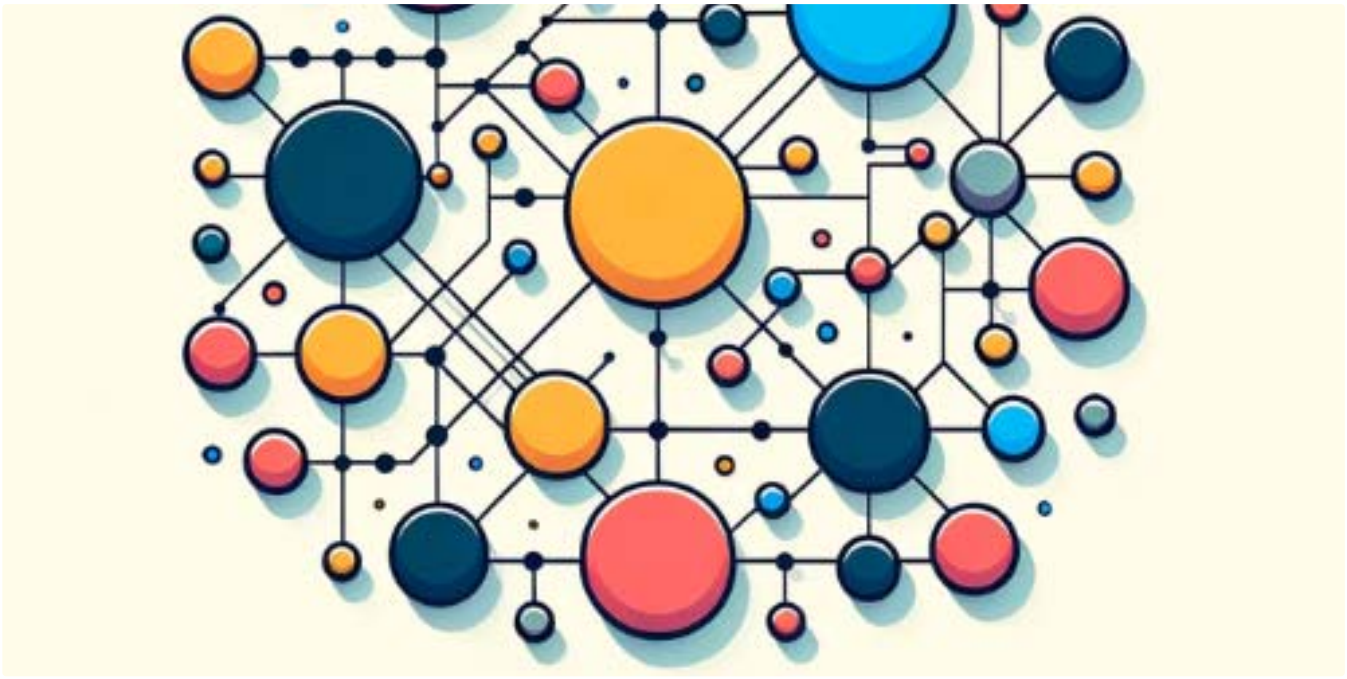
Sudarshan Koirala

Super Easy way to Parse PDFs ✅ 🔥

Introducing LlamaParse from LlamaIndex

2 min read · Mar 9, 2024

8



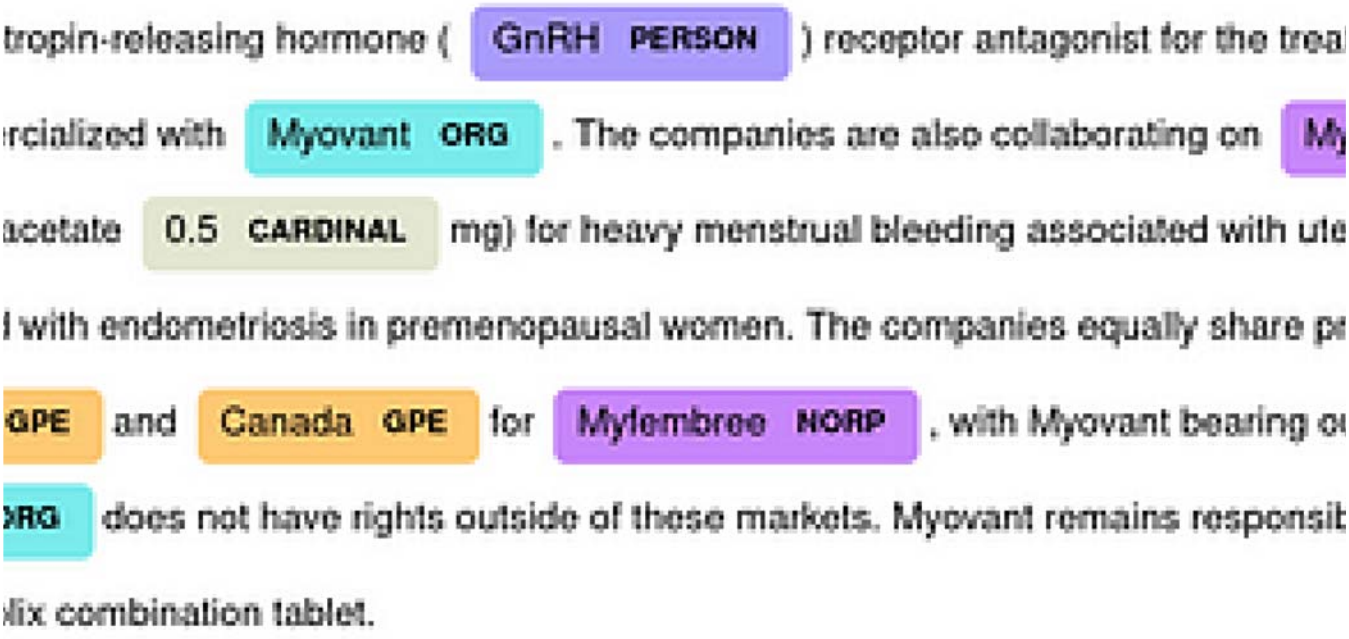
Vardhanam Daga in Stackademic

Why I Chose FalkorDB Knowledge Graph Over Neo4j for Building a RAG Application

A comparison of FalkorDB with other Knowledge Graphs for developing AI applications with.

7 min read · Apr 5, 2024

32



Knowledgator Engineering

Extract any named entities from PDF using custom Spacy pipeline

Portable Document Format (PDF) stands as a predominant file format for distributing content, particularly in academic, scientific...

6 min read · Dec 3, 2023

367

See more recommendations